



Lab 9



Netflix Database

Student Name: Reenad Almadhi

Web development using java with AI integration
Bootcamp

Overview

In this lab, we developed a simplified version of a Netflix database system. The system is designed to manage three core areas: user subscriptions, movie catalogs, and viewing activities. The database consists of 4 main tables:

1. **subscriptions**: Manages pricing plans.
2. **users**: Stores subscriber profiles.
3. **movies**: Catalogs available content.
4. **watch_history**: Tracks which user watched which movie.

We applied specific **attributes**, **constraints**, and **relationships** (Foreign Keys) to ensure data integrity and system efficiency.

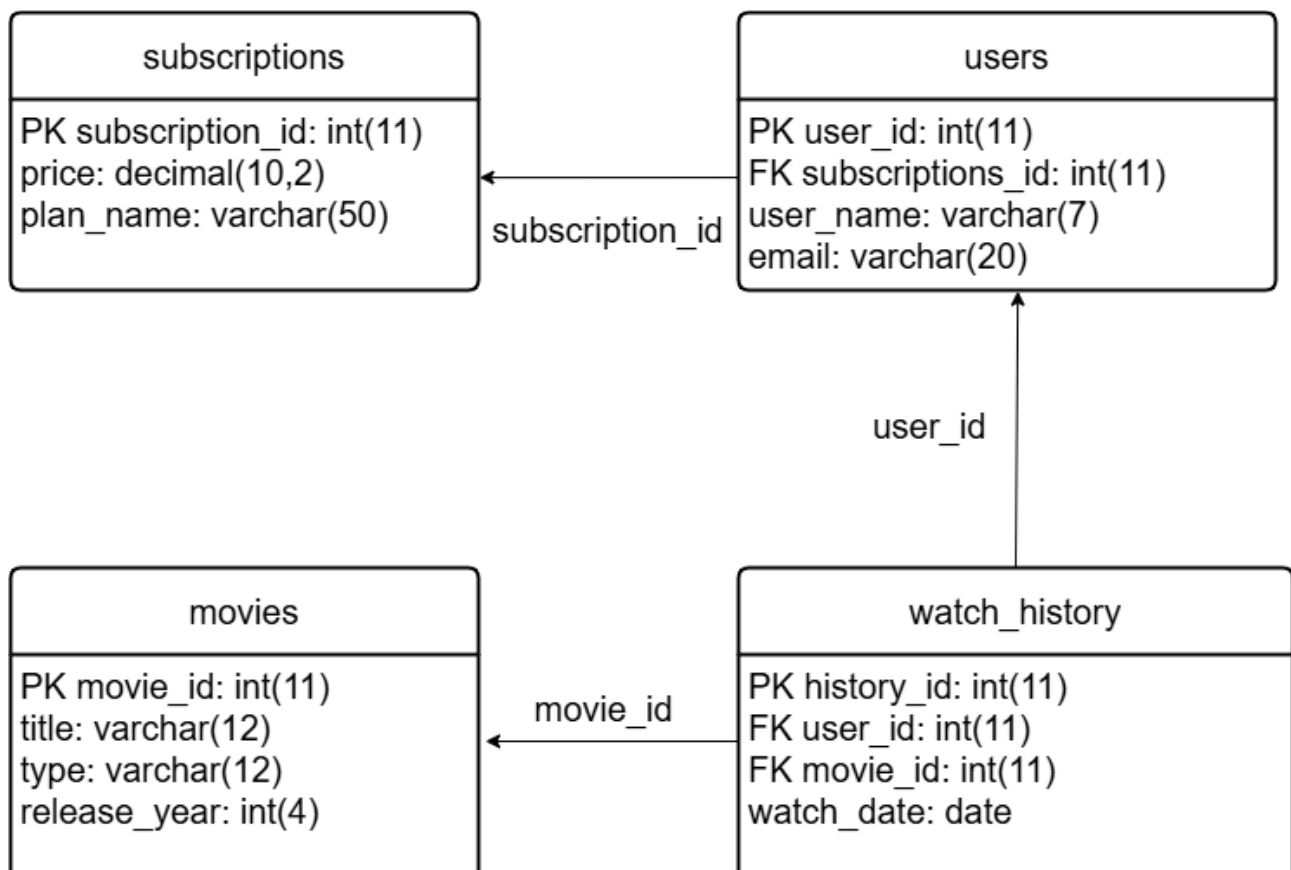
Database Schema

The **Schema** is a visual and logical structure that defines how data is organized and how tables relate to one another. For this project, we provided two types of visualization:

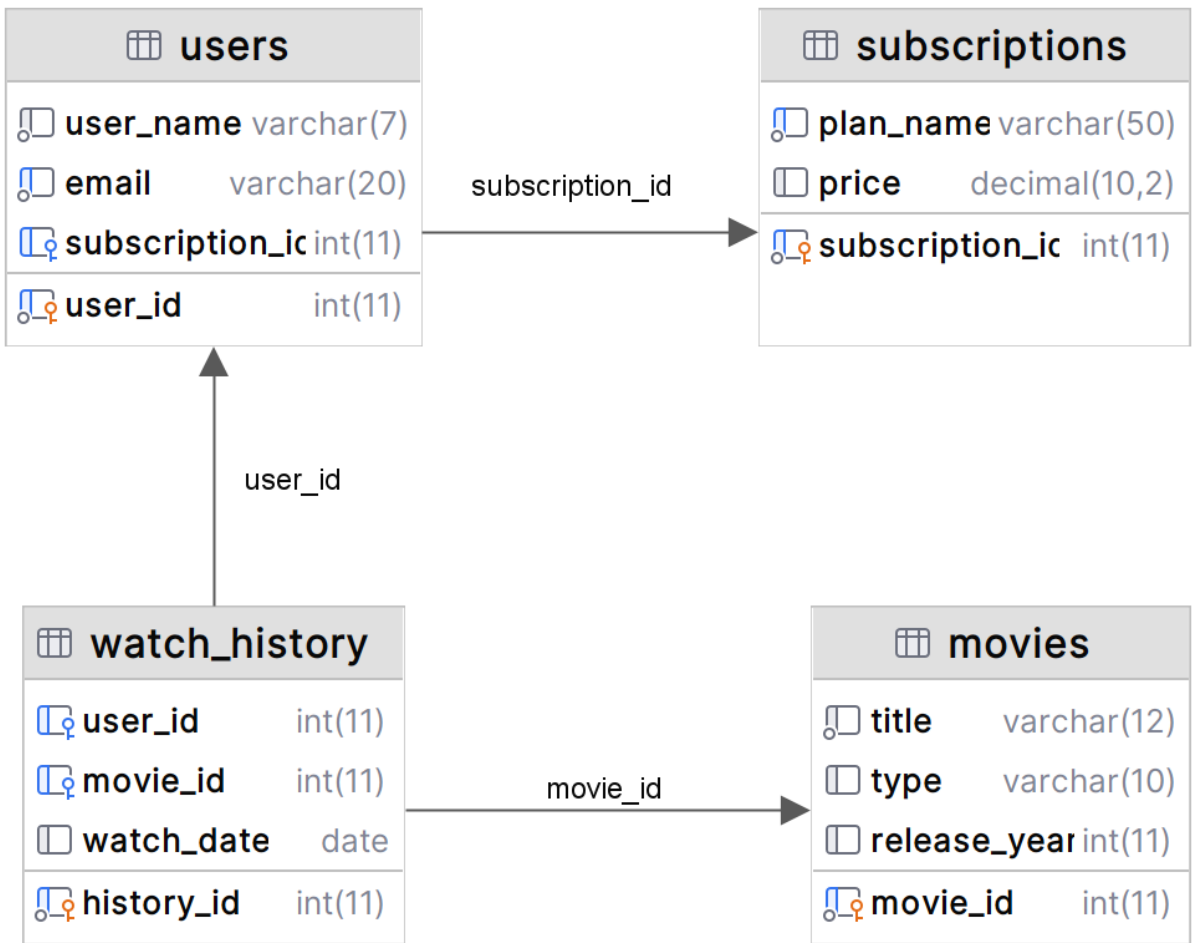
Manual Design (draw.io): A conceptual diagram created to plan the table structures.

Generated Diagram (DataGrip): The actual schema generated by the software, reflecting the final implemented code and physical relationships.

1. Drow.io



2. DataGrip diagram



SQL Code

This section contains the **SQL Script** used to build and manipulate the database. It includes:

DDL (Data Definition Language): To create tables and define constraints.

DML (Data Manipulation Language): To perform **CRUD** operations (Create, Read, Update, Delete) to test the system functionality.

1. DDL

```
create database Netflix;

create table subscriptions (
    subscription_id int primary key,
    plan_name varchar(50) not null unique,
    price decimal(10, 2) default 0
);

create table users (
    user_id int primary key,
    user_name varchar(7) not null,
    email varchar(20) unique not null,
    subscription_id int,
    foreign key (subscription_id) references
subscriptions(subscription_id)
);

create table movies (
    movie_id int primary key,
    title varchar(12) not null,
    type varchar(10),
    release_year int check (release_year > 1888)
);

create table watch_history (
    history_id int primary key,
    user_id int,
    movie_id int,
    watch_date date default current_date,
    foreign key (user_id) references
users(user_id),
    foreign key (movie_id) references
movies(movie_id)
);
```

2. DML

#CRUD Operation

#1 Create

```
insert into subscriptions (subscription_id, plan_name, price) values (1, 'Basic', 9.99),
                                                                    (2, 'Premium', 15.99);
insert into users (user_id, user_name, email, subscription_id) values (101, 'Ahmed',
                                                                    'a@mail.com', 1),
                                                                    (102, 'Sara', 's@mail.com', 2);
insert into movies (movie_id, title, type, release_year) values (501, 'Inception', 'Sci-Fi',
2010);
insert into watch_history (history_id, user_id, movie_id) values (1, 101, 501);
```

#2 Read

```
select * from subscriptions;
select * from users;
select * from movies;
select * from watch_history;
```

#3 Update

```
update subscriptions set price = 19.99 where subscription_id = 2;
update users set user_name = 'Ali' where user_id = 101;
```

#4 Delete

```
delete from movies where movie_id = 501;
```

Read after operation

```
select * from subscriptions;
select * from users;
select * from movies;
select * from watch_history;
```